

Tree Methods I

Sean Hellingman ©

Multivariate Statistics for Applied Data Science (ADSC 4050)

shellingman@tru.ca

Winter 2026



THOMPSON RIVERS UNIVERSITY

Topics

- 2 Recap
- 3 Introduction
- 4 Classification Trees
- 5 Coming Up
- 6 Exercises and References

Recap

- Last time we covered **Quadratic Discriminant Analysis**:
 - Procedure.
 - Interpreting results.
 - Making predictions.
 - **Distance-based Discriminant Analysis**.

Introduction

- Another approach to classification and regression problems is the use of recursive partitioning.
- Unlike regression, these approaches are nonparametric.
- The results are typically easy to interpret and visualize.
 - *May be used to construct random forests and gradient boosting machines.*

Overview

- **Tree-based** methods partition the feature space into a number of smaller (non-overlapping) regions with similar response values using a set of splitting rules.
- There are many methods for constructing decision trees.
 - The **classification and regression tree** (CART) algorithm is the most well known.
- The same approach can be taken for both continuous and categorical response variables.
 - **Classification trees** refer to analyses that use categorical data for the response variable.
 - **Regression trees** refer to analyses that use continuous data for the response variable.

Illustrative Example 1

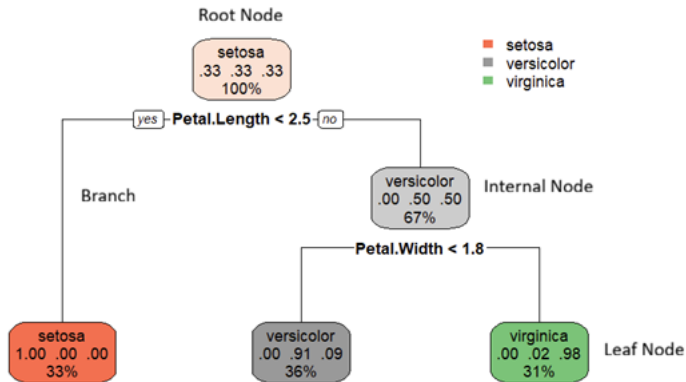
- Run the provided code and examine the results of the classification tree.
- What did you notice?

Classification Trees

Classification Trees

- **Classification trees** refer to analyses that use categorical data for the response variable.
- Procedure:
 - ① Partition the training data into homogeneous subgroups.
 - ② Fit a simple constant in each subgroup.
 - ③ The subgroups (nodes) are formed recursively using binary partitions formed by asking simple yes-or-no questions about each variable.
 - ④ This is done a number of times until a suitable stopping criteria is satisfied.
 - ⑤ The model predicts the output based on the class that has majority representation.

Tree Structure



Partitioning

- CART uses binary recursive partitioning (it's recursive because each split or rule depends on the the splits above it).
- The objective at each node is to find the *best* variable (x_i) to partition the remaining data into one of two regions (R_1 and R_2).
- This is done such that the overall error between the actual response (y_i) and the predicted constant (c_i) is minimized.
- A single variable may be repeated in the process if it continues to provide the best split.

Classification Trees in R

- We can use the *rpart* and the *rpart.plot* packages in R.
 - In Python
- Usage:
 - `tree <- rpart(response ~ X1 + X2 + ... , data = df, method = "class")`
 - `rpart.plot(tree)`

Example 1

- Reconstruct the classification tree from Illustrative Example 1 only using the Sepal Length and Sepal Width.
- Plot your classification tree.
- What do you notice?

Tree Depth

- The tree can continue to *grow* until each object is *correctly* classified.
 - This will lead to overfitting and poor generalization performance.
- There is a balance that needs to be found between complexity and generalization performance.
- Two primary ways this can be done:
 - 1 **Early stopping:** Restrict the tree depth to a certain level or to restrict the minimum number of observations allowed in any terminal node.
 - 2 **Pruning:** Grow a very large, complex tree and then prune it back to find an optimal subtree.

Early Stopping

- **Early stopping** explicitly restricts the growth of the tree.
- The most common approaches are to restrict the tree depth to a certain level or to restrict the minimum number of observations allowed in any terminal node.
- To apply these methods in R:
 - Depth restriction: `maxdepth = k`
 - Minimum in leaf node: `minbucket = 1`

Example 2

- Load the *ptitanic* data from the *rpart* package and do the following:
 - 1 Construct a binary classification tree with `survived` as the response variable.
 - 2 Examine your tree.
 - 3 Adjust the maximum depth of your tree
 - 4 Adjust the minimum number of observations allowed in the leaf nodes.
 - 5 Construct a confusion matrix for each of your trees.
- What do you notice?

Pruning

- **Pruning** is the process of growing a very large, complex tree and then *pruning* it back to find an optimal subtree.
- We can use the following minimization function:

$$\text{minimize}\{SSE + \alpha|T|\} \quad (1)$$

- Similar to regularization problems, the size of α will determine the complexity of the tree.
- Evaluate multiple models across a spectrum of α and use CV to identify the optimal value.

Pruning in R

- The `cp` (complexity parameter) helps to determine the complexity of the trees in R.
 - *Smaller values indicate more complex trees.*
- Construct a complex tree and use cross validation to identify the best value for α :
 - `plotcp(tree)`
 - `tree$cpable[which.min(tree$cpable[, "xerror"]), "CP"]`
 - `bestcp <-`
 `tree$cpable[which.min(tree$cpable[, "xerror"]), "CP"]`
 - `tree.pruned <- prune(tree, cp = bestcp)`

Example 3

- Load the *ptitanic* data from the *rpart* package and do the following:
 - ① Construct a very deep binary classification tree with *survived* as the response variable.
 - ② Examine your tree.
 - ③ Adjust the provided code to prune your complex tree.
 - ④ Examine your new tree.
 - ⑤ Construct a confusion matrix for each of your trees.
- What do you notice?

Pruning in R Cross-Validation

- You can also use the caret package in R to perform cross validation on different values of α :
 - ```
tree <- train(
 Sale_Price ~ .,
 data = df.train,
 method = "rpart",
 trControl = trainControl(method = "cv", number = 10),
 tuneLength = 20
)

ggplot(tree)
```
- Based on the cross validation, you can use the `vip()` function to get an idea of variable importance.

## Example 4

- Load the *ptitanic* data from the *rpart* package and do the following:
  - ① Use the `train()` function from the *caret* package to perform cross-validation on  $\alpha$ .
  - ② Examine the results.
  - ③ Are they very different from Example 3?
  - ④ Identify the most important variables.
- What do you notice?

## Classification Tree Comments

- Classification trees provide the basis of more robust procedures.
- The complexity of the tree is related to your objective.
  - Individual trees generally do not possess sophisticated predictive capabilities.
- Trees require very little pre-processing!
  - We did not transform any of our variables in these examples.

## Next Lecture

- We will discuss **Tree Methods II**.
- Regression trees.
- Predicting with trees.

## Exercise 1

- Import the wine dataset from the *HDclassif* package.
- Take some time to get to know the data (including visualizations).
- Construct an out-of-the box classification tree for this dataset.
- Use early stopping to construct simpler classification trees for this dataset.
- Find the ideal complexity parameter ( $cp$ ) using cross validation and construct this tree.
- Finally, compare the classification accuracy of all of your trees on the training set.
- What do you notice?

## References & Resources

- 1 Bakker, J. D. (2026) *Applied Multivariate Statistics in R*
  - 2 Peng R. D. (2020) *Exploratory Data Analysis with R*
  - 3 Timbers T., Campbell T., and Lee M. (2024) *Data science: A first introduction*. Chapman and Hall/CRC
  - 4 Bradley Boehmke B., and Greenwell B. (2020) *Hands-On Machine Learning with R*. Taylor & Francis
- Trees & Classification
  - CART
  - `rpart()`
  - `rpart.plot()`
  - `vip()`
  - `confusionMatrix()`